

1.Ανάγνωση του αρχείου CSV: Αρχικά, αναγνώστηκε το αρχείο CSV που περιείχε δεδομένα εμπορίου σχετικά με τον COVID-19, όπου τα δεδομένα είχαν ένα πεδίο "Date" και ένα πεδίο "Cumulative". Το αρχείο CSV διαβάστηκε γραμμή προς γραμμή και κάθε γραμμή δεδομένων αποθηκεύτηκε ως αντικείμενο Data σε ένα vector, όπου το Data είναι μια δομή που περιέχει μια string date και ένα long long Cumulative.

2.Ταξινόμηση των Δεδομένων: Μετά την επιτυχή ανάγνωση των δεδομένων σε ένα vector, ταξινομήσαμε αυτό το vector βάσει του πεδίου "Cumulative" χρησιμοποιώντας δύο διαφορετικούς αλγορίθμους: Heap Sort και Quick Sort.

3.Ανάλυση Επιδόσεων:

Heap Sort: Ο Heap Sort ολοκληρώθηκε σε περίπου 56191 μικροδευτερόλεπτα (ή 0,056 δευτερόλεπτα). Ο Heap Sort είναι ένας αλγόριθμος ταξινόμησης με χρονική πολυπλοκότητα $O(n \log n)$ και στη χειρότερη περίπτωση, γεγονός που τον καθιστά αποδοτικό για μεγάλα σύνολα δεδομένων.

Quick Sort: Ο Quick Sort ολοκληρώθηκε σε περίπου 2.960.998 μικροδευτερόλεπτα (ή περίπου 2,96 δευτερόλεπτα). Ο Quick Sort είναι επίσης ένας αλγόριθμος ταξινόμησης με χρονική πολυπλοκότητα $O(n \log n)$ στην καλύτερη και στη μέση περίπτωση, αλλά η χρονική πολυπλοκότητα μπορεί να αυξηθεί σε $O(n^2)$ στη χειρότερη περίπτωση. Αυτό μπορεί να εξηγήσει γιατί ο Quick Sort ήταν πιο αργός σε αυτό το συγκεκριμένο σύνολο δεδομένων σε σύγκριση με τον Heap Sort.

4.Συμπέρασμα: Το Heap Sort αποδείχτηκε πιο γρήγορο από τον Quick Sort για αυτό το συγκεκριμένο σύνολο δεδομένων. Επιπλέον, κατά την εφαρμογή του αλγορίθμου Quick Sort, αντιμετωπίσαμε ένα πρόβλημα με stack overflow καλώντας αναδρομικά τη συνάρτηση Quick Sort. Αυτό το πρόβλημα λύθηκε υλοποιώντας τον Quick Sort επαναληπτικά(iteratively) αντί για αναδρομικά(recursively). Αυτό καταδεικνύει την σημασία της επιλογής του κατάλληλου αλγορίθμου για τα δεδομένα μας, αλλά και της προσαρμογής του αλγορίθμου βάσει των συγκεκριμένων συνθηκών εκτέλεσης.